SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Mode: In Class

Total Marks:50

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

192321050

I [illegible] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sno	Problem Understanding (20)	Method Selection (20)	Solution Process (20)	Accuracy of Calculation (20)	Interpretation of Results(10)	Total Score (out of 100)
1	/4	/4	/4	/4	/4	
2	/4	/4	/4	/4	/4	
3	/4	/4	/4	/4	/4	
4	/4	/4	/4	/4	/4	
5	/4	/4	/4	/4	/4	
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	
	/20	/20	/20	/20	/10	/100

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Trace process of input expression P=(a*b)+(b*c)*2

Compile phase for P=(a*b)+(b*c)*2

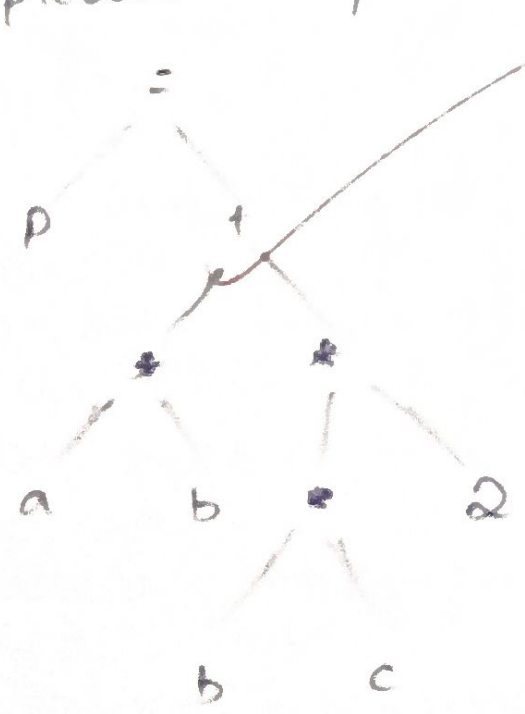
a) Lexical Analysis [tokens].

The lexical analyzer scans the stream of characters and stripe whitespace's, yeilding the tokens

* identifiers : id[P], id[a], id[b], id[c]
* Assignment Operator : =
* Arithmetic Operators : +, *, *
* parathesis : (,) , (,).
* Constant / Integral Literal : 2

b) Syntax tree

Operator precedence plays multiplication above condition.



84/100

11/03/22

ex Intermediate Code generation [three address code]

assembly real / floating point representation of general register type

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$t_4 = t_1 + t_3$

$p = t_4$.

ex intermediate Code optimization

→ algebraic transformation : if value are integers multiplication by 2

$t_1 = a * b$

$t_2 = (b * c) * 2$

$p = t_1 + t_2$

ex Significance of Final target Code :

The target code generated maps the optimized intermediate representation into machine. (eg: R0, R1, MUL, ADD). It's primary significance is ensuring hardware execution efficiency and instruction level parallelism

... least one 1:

(0+1)* 1 (0+1)*

iii. Both numbers of 0's and 1's are odd:

(0+1)* 1 (0+1)* 1 (0+1)* | 0* 1 0* 1 (0+1)*

strings recognized by the 4 state DFA tracking parity [odd, odd]:

(aa+ab)(bb)* ba)* a(b+ba(bb)* a)(bb)* (b+a)(aa)* ab)

where a = 0, b = 1

iv. starts with 1 and ends with 0:

1(0+1)*0

v. do not contain consecutive 1's ("no '11'"):

(01|0)* (1|ε) or (10·10)* (1|ε)

* Lexical Analyzer for simplified language.

int var123 = 12345; float var456 = 123.45;

if (var123 == var456) {return;}

a) Number of tokens:

* there are 20 tokens.

int , var123 , = , 12345 , float , var456 , = , 123.45 ,

if , (, var123 , == , var456 ,) , { , return , ; , } .

3. C Tokenization & Lexical Errors.

i) Token Count analysis

```
main()                          //main,()          -> 3 tokens
{                               //{                -> 1 token
int a=10;                       //main,int,a,=,10,; -> 5 tokens
char b='abc';                   //char,b,=,'abc,'; -> 5 tokens
int t c=30;                     //int,t,c,=,30,;   -> 6 tokens
ch ar d='xyz";                  //ch,ar,d,=,xyz,;  -> 6 tokens
in t m=40.5;                    //in,t,m,=,40.5,;  -> 6 tokens
}                               //{                -> 1 token
```

∴ total tokens = 33.

ii) Lexical errors :

x = 1xab; contains a lexical error. in C, an identifier cannot begin with a digit and 1xab is neither a valid numeric constant nor a valid hex literal.

Regular expressions over $\Sigma = \{0, 1\}$.

i) even number of 0's :

$$1^*(01^*01^*)^*$$

b) categorized list of tokens

* keyword [4]
* identifiers [4]
* operators [3]
* Integral literals [3]
* Floating-point literals [3]
* separators [7]

c) FSM state transition estimation.

in a standard finite automaton form]:

→ character transition: each character consumed within a token requires 1 transition.

→ Delimiter/Lookahead: Identifiers, identifiers and keywords required 1 transition.

→ whitespace transition: each whitespace character consumes 1 transition from start state back to itself.